

TAARA

Infrastructure Intelligence Platform

Prevent Crash, Preserve Cash

/ 100 — Repository Risk Score

100

Field	Value
Client	OWASP NodeGoat Demo
Repository	https://github.com/OWASP/NodeGoat
Scan Date	2026-05-17
Packages Analysed	1479
Total Findings	165 (46 critical, 63 high)
Quantum Fidelity F	0.428 — UNSAFE — dependency graph far from secure baseline

This report is confidential. All findings are derived from automated analysis of real data. No issues have been invented or extrapolated.

1. Executive Summary

Metric	Value	Status
Repository Risk Score	100/100	CRITICAL
Critical Findings	46	Fix within 24 hours
High Findings	63	Fix within 1 week
Medium Findings	56	Fix within 1 month
Packages Scanned	1479	Live OSV.dev lookup
Security Posture Fidelity F	0.428	FAR from secure baseline

What This Means

On May 17, 2026, our security team scanned the OWASP NodeGoat Demo website, specifically the code repository located at <https://github.com/OWASP/NodeGoat>, and found a total of 165 security vulnerabilities, including 46 critical issues, in the 1479 packages that were examined. The most dangerous packages identified include growl, underscore, set-value, handlebars, and npm, which are all widely used components that can be exploited by attackers to gain unauthorized access or control. If these vulnerabilities are not addressed, a malicious actor could potentially inject commands, execute arbitrary code, or pollute prototypes, allowing them to take control of the application, steal sensitive data, or disrupt service, which could lead to significant financial and reputational damage. The most urgent action to take today is to immediately update or patch the growl package to a version later than 1.10.0, as it is currently vulnerable to command injection attacks, which poses an immediate and significant risk to the security of the application.

2. Findings

46 CRITICAL · 63 HIGH · 56 MEDIUM · 0 LOW · Total: 165

Quantum Fidelity: F=0.428 — this repository's security posture is nearly orthogonal to a secure baseline. An attacker exploiting any of these vulnerabilities faces almost no security controls pointing in the right direction.

Dependency Risk Analysis (AI-generated from real findings)

The interdependencies in this application create a significant risk for users, as an attacker could exploit vulnerabilities such as prototype pollution in `async` and `minimist`, deserialization of untrusted data in `bson`, and code injection in `fsevents`. If exploited, these vulnerabilities could allow an attacker to inject malicious code, overwrite arbitrary files, or read sensitive data. The most urgent packages to fix are `async`, `bson`, and `fsevents`, as they have direct exploit chains to the application and could allow for remote code execution or data tampering. The presence of transitive dependencies, such as `tar`, which has multiple critical vulnerabilities, further amplifies the risk, as an attacker could potentially exploit a vulnerability in a nested dependency to gain access to the application.

Analysis alignment score: F=0.508 — Answer partially aligned with policy

2.1 Critical Findings — Fix Immediately

Package	Version	CVE ID	Fix Version	Description
growl	1.9.2	GHSA-qh2h-chj9-jff	1.10.0	GHSA-qh2h-chj9-jffq: Growl before 1.10.0 vulnerable to
underscore	1.8.3	GHSA-cf4h-3jhx-xvh	1.12.1	GHSA-cf4h-3jhx-xvhq: Arbitrary Code Execution in unders
set-value	2.0.0	GHSA-4g88-fppr-53p	2.0.1	GHSA-4g88-fppr-53pp: Prototype Pollution in set-value
handlebars	4.0.5	GHSA-w457-6q6x-cgp	4.3.0	GHSA-w457-6q6x-cgp9: Prototype Pollution in handlebars
npm	3.10.10	GHSA-ph34-pc88-72g	5.7.1	GHSA-ph34-pc88-72gc: Incorrect Permission Assignment fo
shelljs	0.3.0	GHSA-4rq4-32rv-6wp	0.8.5	GHSA-4rq4-32rv-6wp6: Improper Privilege Management in s
json-schema	0.2.3	GHSA-896r-f27r-55m	0.4.0	GHSA-896r-f27r-55mw: json-schema is vulnerable to Proto
getobject	0.1.0	GHSA-957j-59c2-j69	1.0.0	GHSA-957j-59c2-j692: Prototype pollution in getobject
lodash	4.17.11	GHSA-p6mc-m468-83g	4.17.19	GHSA-p6mc-m468-83gw: Prototype Pollution in lodash
mixin-deep	1.3.1	GHSA-fhjf-83wg-r2j	1.3.2	GHSA-fhjf-83wg-r2j9: Prototype Pollution in mixin-deep
npm	3.10.10	GHSA-4328-8hgf-7wj	6.13.4	GHSA-4328-8hgf-7wjr: npm Vulnerable to Global node_modu
babel-traverse	6.11.4	GHSA-67hx-6x53-jw9	—	GHSA-67hx-6x53-jw92: Babel vulnerable to arbitrary code
ms	0.7.1	GHSA-w9mr-4mfr-499	2.0.0	GHSA-w9mr-4mfr-499f: Vercel ms Inefficient Regular Expr
minimist	1.2.0	GHSA-xvch-5gv4-984	1.2.6	GHSA-xvch-5gv4-984h: Prototype Pollution in minimist
handlebars	4.0.5	GHSA-765h-qjxv-5f4	4.7.7	GHSA-765h-qjxv-5f44: Prototype Pollution in handlebars
bson	1.0.9	GHSA-v8w9-2789-6hh	1.1.4	GHSA-v8w9-2789-6hhr: Deserialization of Untrusted Data
handlebars	4.0.5	GHSA-xjpj-3mr7-gcp	4.7.9	GHSA-xjpj-3mr7-gcpf: Handlebars.js has JavaScript Injec
handlebars	4.0.5	GHSA-2w6w-674q-4c4	4.7.9	GHSA-2w6w-674q-4c4q: Handlebars.js has JavaScript Injec
hoek	2.16.3	GHSA-jp4x-w63m-7wg	5.0.3	GHSA-jp4x-w63m-7wgm: Prototype Pollution in hoek

Package	Version	CVE ID	Fix Version	Description
async	2.6.1	GHSA-fwr7-v2mv-hh2	3.2.2	GHSA-fwr7-v2mv-hh25: Prototype Pollution in async

... and 26 more critical findings.

2.2 High Findings — Fix This Week

Package	Version	CVE ID	Fix Version	Description
minimatch	3.0.4	GHSA-7r86-cg39-jmm	10.2.3	GHSA-7r86-cg39-jmmj: minimatch has ReDoS: matchOne() co
underscore	1.8.3	GHSA-qpx9-hpmf-5gm	1.13.8	GHSA-qpx9-hpmf-5gmw: Underscore has unlimited recursion
hawk	3.1.3	GHSA-44pw-h2cw-w3v	9.0.1	GHSA-44pw-h2cw-w3vq: Uncontrolled Resource Consumption
kind-of	6.0.2	GHSA-6c8f-qphg-qjg	6.0.3	GHSA-6c8f-qphg-qjgp: Validation Bypass in kind-of
minimatch	3.0.4	GHSA-f8q6-p94x-37v	3.0.5	GHSA-f8q6-p94x-37v3: minimatch ReDoS vulnerability
semver	5.7.0	GHSA-c2qf-rxjj-qgg	7.5.2	GHSA-c2qf-rxjj-qggw: semver vulnerable to Regular Expre
minimatch	3.0.4	GHSA-23c5-xmqv-rm7	10.2.3	GHSA-23c5-xmqv-rm74: minimatch ReDoS: nested *() extglo
i	0.3.6	GHSA-x55w-vjjp-222	0.3.7	GHSA-x55w-vjjp-222r: inflect vulnerable to Inefficient
cross-spawn	7.0.3	GHSA-3xgq-45jj-v27	7.0.5	GHSA-3xgq-45jj-v275: Regular Expression Denial of Servi
ansi-regex	3.0.0	GHSA-93q8-gq69-wqm	6.0.1	GHSA-93q8-gq69-wqmw: Inefficient Regular Expression Com
handlebars	4.0.5	GHSA-2qvq-rjwj-gvw	4.7.9	GHSA-2qvq-rjwj-gvw9: Handlebars.js has Prototype Pollut
lodash	4.17.21	GHSA-f23m-r3pf-42r	4.18.0	GHSA-f23m-r3pf-42rh: lodash vulnerable to Prototype Pol
moment	2.24.0	GHSA-8hfj-j24r-96c	2.29.2	GHSA-8hfj-j24r-96c4: Path Traversal: 'dir/../../filenam
send	0.16.2	GHSA-m6fv-jmcg-4jf	0.19.0	GHSA-m6fv-jmcg-4jfg: send vulnerable to template inject
qs	5.2.1	GHSA-6rw7-vpxm-498	6.14.1	GHSA-6rw7-vpxm-498p: qs's arrayLimit bypass in its brac

... and 48 more high findings.

2.3 CI/CD and Infrastructure Findings

[CRITICAL] EOL runtime: nodejs 12 (1478 days past EOL)

node:12-alpine uses nodejs 12, which reached end-of-life on 2022-04-30 (1478 days ago). No security patches are being issued. Latest supported version: 12.22.12.

Fix: See TAARA recommendation

[CRITICAL] EOL runtime: nodejs 12 (1478 days past EOL)

node:12-alpine uses nodejs 12, which reached end-of-life on 2022-04-30 (1478 days ago). No security patches are being issued. Latest supported version: 12.22.12.

Fix: See TAARA recommendation

[CRITICAL] Action pinned to floating tag: actions/cache@v2

.github/workflows/e2e-test.yml line 26: 'actions/cache@v2' uses a tag, not a commit SHA. The tag can be moved to point at malicious code. This is exactly how the tj-actions attack worked in March 2025, compromising 23,000+ repositories. Fix: pin to the full commit SHA — uses: actions/cache

Fix: See TAARA recommendation

[CRITICAL] Action pinned to floating tag: actions/setup-node@v1

.github/workflows/e2e-test.yml line 21: 'actions/setup-node@v1' uses a tag, not a commit SHA. The tag can be moved to point at malicious code. This is exactly how the tj-actions attack worked in March 2025, compromising 23,000+ repositories. Fix: pin to the full commit SHA — uses: actions/

Fix: See TAARA recommendation

[CRITICAL] Action pinned to floating tag: actions/checkout@v2

.github/workflows/e2e-test.yml line 16: 'actions/checkout@v2' uses a tag, not a commit SHA. The tag can be moved to point at malicious code. This is exactly how the tj-actions attack worked in March 2025, compromising 23,000+ repositories. Fix: pin to the full commit SHA — uses: actions/ch

Fix: See TAARA recommendation

[CRITICAL] Action pinned to floating tag: actions/checkout@v2

.github/workflows/lint.yml line 16: 'actions/checkout@v2' uses a tag, not a commit SHA. The tag can be moved to point at malicious code. This is exactly how the tj-actions attack worked in March 2025, compromising 23,000+ repositories. Fix: pin to the full commit SHA — uses: actions/checko

Fix: See TAARA recommendation

[CRITICAL] Action pinned to floating tag: actions/upload-artifact@v2

.github/workflows/e2e-test.yml line 58: 'actions/upload-artifact@v2' uses a tag, not a commit SHA. The tag can be moved to point at malicious code. This is exactly how the tj-actions attack worked in March 2025, compromising 23,000+ repositories. Fix: pin to the full commit SHA — uses: act

Fix: See TAARA recommendation

[CRITICAL] Action pinned to floating tag: actions/setup-node@v1

.github/workflows/lint.yml line 21: 'actions/setup-node@v1' uses a tag, not a commit SHA. The tag can be moved to point at malicious code. This is exactly how the tj-actions attack worked in March 2025, compromising 23,000+ repositories. Fix: pin to the full commit SHA — uses: actions/setu

Fix: See TAARA recommendation

2.4 Exploit Chains — How an Attacker Gets In

Each chain shows the path from your application to a vulnerable package. Score = severity × proximity. Score 10 = direct critical vulnerability.

Attack Path	Score	CVE	Fix
your application → async@2.6.1	10.0	GHSA-fwr7-v2mv-hh2	3.2.2
your application → bson@1.0.9	10.0	GHSA-v8w9-2789-6hh	1.1.4
your application → ms@0.7.1	10.0	GHSA-w9mr-4mfr-499	2.0.0
your application → minimist@1.2.0	10.0	GHSA-xvch-5gv4-984	1.2.6
your application → fsevents@1.2.9	10.0	GHSA-8r6j-v8pm-fqw	1.2.11
your application → tar@4.4.8	10.0	GHSA-3jfq-g458-7qm	3.2.2
your application → tar@4.4.8	10.0	GHSA-5955-9wpr-37j	4.4.18
your application → tar@4.4.8	10.0	GHSA-83g3-92jg-28c	7.5.8

3. Prioritised Action Plan

Generated from your actual findings:

WEEK 1

This week, we will upgrade the Growl package to version 1.10.0 or later using the command ``npm install growl@^1.10.0`` to mitigate the Command Injection vulnerability, update the underscore package to the latest version using ``npm install underscore@latest`` to prevent Arbitrary Code Execution, and update the json-schema package using ``npm install json-schema@latest`` to fix the Prototype Pollution vulnerability.

MONTH 1

This month, we will update the handlebars package to the latest version using ``npm install handlebars@latest`` to prevent Prototype Pollution, review and update the permissions in the NP package to ensure Correct Permission Assignment for Critical Resources, and update the shelljs package using ``npm install shelljs@latest`` to fix Improper Privilege Management.

QUARTER

We will establish an ongoing practice of regularly reviewing the dependencies in our package.json file and updating them to the latest versions, particularly for packages like minimatch, to prevent ReDoS vulnerabilities, and implement a code review process to detect and prevent vulnerabilities like Prototype Pollution in packages such as getobject.

4. What Continuous Monitoring Would Have Caught

This report is a point-in-time snapshot. The findings in it existed before today — some of these CVEs have been public for years. Without continuous monitoring, the gap between a vulnerability existing and your team knowing about it can be months.

	Point-in-Time Audit (this report)	TaaraWare Continuous Monitoring
Detection lag	Months — until someone runs a scan	Under 10 minutes from occurrence
CVE coverage	Packages present at scan time	Every new CVE published, checked against your stack automatically
Config drift	Not detected between scans	sshd_config changes, new open ports, Dockerfile changes — alerted
CI/CD tampering	Caught only if re-scanned	GitHub Actions changes flagged on push
Cost of missing it	Breach discovered after the fact	Behavioural anomaly flagged before data leaves

This scan found **46 critical vulnerabilities** across 165 findings. At the IBM Cost of Data Breach India 2024 baseline of **₹2–5 Cr** for MSMEs, a single breach from one of these 46 critical issues costs more than years of continuous monitoring. TaaraWare is deployed as a lightweight agent on your server — it collects, TAARA analyses, you approve high-impact actions.

What TaaraWare Does Automatically

Action	Requires Human Approval?
Block SSH brute-force IP (>50 failures, 0 success)	No — pre-approved policy
Alert on new public port opened	No — automated alert
Alert on config file change (sshd_config, Dockerfile)	No — automated alert
New critical CVE matches your installed packages	No — automated alert within 10 min
Rotate production secrets	Yes — always requires approval
Delete any resource	Yes — always requires approval
Modify CI/CD pipeline	Yes — always requires approval

The difference between a point-in-time audit and continuous monitoring is the difference between finding a breach after and preventing it before.

To deploy TaaraWare: taara.in/deploy | hello@taara.in

Data Sources & Scope

Scan target: <https://github.com/OWASP/NodeGoat>

Date: 2026-05-17

Packages checked: 1479 (via OSV.dev live API)

Data sources: OSV API (api.osv.dev), endoflife.date API, KB GraphRAG, lockfile parsing, dependency graph (networkx), GitHub Actions pattern analysis

Limitations: Point-in-time assessment — security posture changes continuously. OSV CVE data depends on package ecosystem coverage. This report does not constitute a penetration test. Recommendations should be reviewed by a qualified professional before implementation.

*Generated by TAARA Infrastructure Intelligence Platform | Prevent Crash, Preserve Cash
Report date: 2026-05-17 03:32 | TAARA Q.0*